

Trainable Vector Quantization for Large Language Models

Anonymous ACL submission

Abstract

Vector Quantization (VQ) is a rapidly evolving technique for compressing Neural Networks, specifically Large Language Models, reducing memory usage and accelerating inference. However, optimizing vector-quantized models remains challenging due to the discrete nature of their parameters. Existing methods rely on training-free, quantization-friendly structural modifications and discrete optimization algorithms to improve model quality. We propose Trainable Vector Quantization (TVQ), a novel approach that leverages Straight-Through Estimation to enable gradient-based optimization of vector-quantized models. By reformulating VQ optimization with trainable latent weights and using an efficient approximate nearest neighbor search, TVQ allows the optimization of arbitrary loss functions for vector-quantized weights. TVQ is simple to implement using standard deep learning frameworks and achieves state-of-the-art performance on LLAMA 2, MISTRAL, and LLAMA 3 models.

1 Introduction

Recent advancements in machine learning have led to significant progress in Natural Language Processing, particularly in text generation and question answering (Achiam et al., 2023; Dubey et al., 2024; Guo et al., 2025). A major factor driving this progress is the rapid increase in Neural Network (NN) size, which leads to enhanced performance but also raises computational costs and memory consumption, fueling demand for efficient NNs.

A variety of compression and acceleration techniques exist, including speculative decoding (Leviathan et al., 2023), quantization (Hubara et al., 2018), pruning (Han et al., 2015), knowledge distillation (Geoffrey Hinton and Dean, 2014) and Neural Architecture Search (Zoph, 2016). Among them, quantization stands out as a technique that can be applied post-training with relatively low computational (GPU) requirements (Li et al., 2021),

while most others must be applied during pre-training. It achieves memory reduction and accelerates inference by lowering the precision of NN parameters – for instance, by using 8 or lower-bit number format instead of the default 32-bit format.

A research direction in quantization, known as Vector Quantization (VQ), has gained traction with the rise of Large Language Models (LLM), demonstrating significant memory reduction advantages over traditional scalar quantization in low-bit settings (Egiazarian et al., 2024; Tseng et al., 2024; Liu et al., 2024). In contrast to scalar quantization, which encodes individual entries of the weight tensor with a fixed set of scalar values, VQ represents chunks of the weight tensor using a fixed set of vectors. Naturally, after applying VQ, the model quality deteriorates and needs to be recovered by subsequent VQ-aware optimization. Here lies the main challenge of VQ research – optimization of an NN composed of a set of unique vectors. Standard gradient descent optimization methods are not directly applicable to Vector Quantized models because most of their parameters are discrete. To recover model quality, prior work has explored various techniques, such as outlier suppression (Liu et al., 2024), principal component extraction (Leconte et al., 2024), column permutation (Leconte et al., 2024), and data-aware initialization (Liu et al., 2024). Others have employed training-free algorithms, including beam search (Egiazarian et al., 2024) and discrete first and second-order optimization (Tseng et al., 2024; Liu et al., 2024). However, these approaches can not train Vector-Quantized models to minimize an arbitrary loss function, which is the primary source of learning signals for NNs.

A similar challenge exists in other compression techniques, such as scalar quantization and pruning, both of which involve discrete variable optimization. Quantization maps values to discrete integers, while pruning eliminates values be-

low a certain threshold by rounding them to zero, making zero a discrete state. In both these areas, the state-of-the-art (SotA) approaches leverage Straight-Through Estimation (STE) (Bengio et al., 2013; Esser et al., 2020; Vanderschueren and De Vleeschouwer, 2023) in different ways to perform gradient descent-based optimization, which appears to be a very powerful technique in practice.

Inspired by these approaches, we introduce a novel method for training Vector-Quantized Neural Networks that leverage STE. For this purpose, we reformulate VQ optimization by introducing trainable continuous latent weights and the “rounding to nearest neighbor” operator. One challenge we encountered was the computational expense of the nearest neighbor search, particularly with large vector sets, which we addressed by incorporating an approximate nearest neighbor search. As a result, our method – referred to as TVQ – enables an efficient first-order optimization of arbitrary loss functions via standard gradient descent for models incorporating vector-quantized tensors.

Besides achieving SotA compression results, we believe a major advantage of our approach lies in the simplicity of implementation. We found that existing methods are often overly complex, involving multiple optimization and preprocessing stages, which can be cumbersome to integrate into existing codebases. In contrast, our approach consists of only two training phases, both of which can be implemented using standard deep learning libraries such as PyTorch (Paszke et al., 2019) and TensorFlow (Abadi et al., 2016), conveniently leveraging pre-implemented optimizers, training loops, and other common tools.

In summary, our contributions are the following:

- We propose TVQ, the first method that enables gradient descent-based optimization of arbitrary loss functions for vector-quantized Neural Networks, which is efficient and simple in implementation.
- In extensive experiments on state-of-the-art LLMs, such as LLAMA 2 (Touvron et al., 2023), MISTRAL (Jiang et al., 2023), and LLAMA 3 (Dubey et al., 2024) we show that TVQ outperforms existing techniques. In addition, we provide detailed ablation studies of our method’s components.

2 Preliminaries

2.1 Vector Quantization

Vector Quantization is a method for compressing a large set of vectors. It groups vectors into clusters, assigning each cluster a representative vector and an index. The collection of representative vectors is called a codebook. Compression is achieved by replacing the original vectors with their corresponding cluster indices and storing only indices and the codebook instead of the original set, significantly reducing memory usage. A process that we refer to as dequantization reconstructs an approximation of the original set by mapping stored indices back to the corresponding vectors from the codebook.

Recent works on LLM compression employed VQ to compress linear layer weight matrices, as they contain the majority of LLM’s parameters. Similar to prior works (Egiazarian et al., 2024; Liu et al., 2024; Tseng et al., 2024), we transform a linear layer weight into a list of vectors to compress. Then, our proposed algorithm searches for the optimal codebook and the indices of representative vectors, which are used as compressed representations of original weight matrices. In more detail, we describe our methods in Section 4.

2.2 Straight Through Estimation

Many scientific problems involve the optimization of functions with discrete values, for instance, quantized integer values in quantization, trajectories in SuperNets used for Neural Architecture Search, and deterministic policies for discrete action spaces in Reinforcement Learning. Applying discrete optimization for such problems is often computationally expensive. For this reason, methods (for example, Dong and Yang, 2019; Jang et al., 2017; Bengio et al., 2013) employ gradient optimization of continuous parameters of functions that return discrete values. These functions, such as rounding and sampling from discrete distribution, have undefined or zero gradients, which results in no parameter updates. STE (Bengio et al., 2013) solves this problem by approximating undefined or zero gradients. Despite not using true gradients, it achieves efficient optimization.

For instance, PACT (Choi et al., 2018) and LSQ (Esser et al., 2020) introduced STE-based backpropagation for scalar quantization, which is still at the heart of all SotA scalar quantization techniques (Chen et al., 2024; Wei et al., 2022). STE-based scalar quantization works as follows. During

training, a quantization function is applied to the weights in the forward pass, simulating the inference behavior and allowing the model to learn with quantization awareness. However, the quantization function, which produces discrete values, includes rounding, so in the backward pass, its analytical derivative is zero almost everywhere, which prevents learning. To address this, STE treats rounding as an identity function during backpropagation, and approximates the derivative of the rounding function with 1, thus passing the gradients through rounding unchanged and enabling gradient-based optimization of quantization parameters.

3 Related Work

Quality degradation induced by VQ can be reduced in different ways. The first one is an optimization of the codebooks and indices of representative vectors from codebook. Codebooks can be optimized straightforwardly via gradient descent – an approach employed in several existing works (Egiazarian et al., 2024; Liu et al., 2024). In contrast, there are no gradient-decent-based methods for optimizing indices. Since indices constitute the overwhelming majority of a vector-quantized model’s parameters (with only a small fraction allocated to codebooks), optimizing only the codebooks yields limited improvements. Consequently, existing works propose various algorithms for indices optimization. For example, Quip# (Tseng et al., 2024) and VPTQ (Liu et al., 2024) split weight matrix into blocks of vectors and use simple Nearest Neighbor strategy to assign indices block-by-block, updating quantization error to the unquantized part of the weight matrix. AQLM (Egiazarian et al., 2024) utilizes discrete optimization technique, specifically, beam search. VPTQ proposes data-aware initialization of the codebook with a Hessian-weighted KMeans algorithm.

The second way to improve VQ is weight or activation preprocessing, which makes the model more quantization-friendly. Quip# suppress outliers in weights incorporating a multiplication by Hadamard matrices into the model architecture. ReALLM (Leconte et al., 2024) proposes weight preprocessing that includes NF-normalization (Dettmers et al., 2024) and column permutations.

The last way is to use a more sophisticated Vector Quantization framework. VPTQ and

CRVQ (Xu et al., 2024) store important (salient) weights in an additional codebook and incorporate sensitivity metrics to measure weight saliency. ReALLM tunes an additional low-rank matrix, similar to low-rank LLM adapters (Hu et al., 2022; Dettmers et al., 2024), compresses weight before VQ using encoder NN and stores decoder NN for dequantization.

In all mentioned works (except for AQLM), proposed structural modifications significantly contribute to competitive performance and complicate model inference. In contrast, our research targets the development of a powerful optimization algorithm for indices, illustrating that a simple VQ framework can achieve competitive performance with effective optimization.

4 Method

During VQ we split the rows of the weight matrix $W \in \mathbb{R}^{n_{out} \times n_{in}}$ into d -dimensional vectors $w_{ij}^d \in \mathbb{R}^d$ forming a set of $n_{out} \cdot \frac{n_{in}}{d}$ vectors:

$$W = \left\{ w_{ij}^d \right\}_{i=1, j=1}^{n_{out}, \frac{n_{in}}{d}}$$

These vectors are quantized using codebook $C \in \mathbb{R}^{2^B \times d}$ and $n_{out} \times \frac{n_{in}}{d}$ matrix I containing indices for each vector w_{ij}^d . Matrix I consists of integer B -bit indices to encode 2^B vectors $c_k^d \in \mathbb{R}^d$ from the codebook C that are called codewords. The goal is to find the optimal codebook C and indices matrix I that are used as a compressed representation of the weight W .

4.1 Vector Quantization with Straight Through Estimation

For model inference, the weights are reconstructed from compressed representation via the dequantization procedure. Dequantized weight $\widehat{W}$ is obtained by replacing each index from the indices matrix I with the corresponding vector from the codebook C :

$$\widehat{W} = \left\{ \hat{w}_{ij}^d \mid \hat{w}_{ij}^d = C_{I_{ij}} \right\}_{i=1, j=1}^{n_{out}, \frac{n_{in}}{d}} = C_I \quad (1)$$

According to (1) dequantized weight $\widehat{W}$ depends on the parameters C and I . The indices stored in I cannot be directly optimized with gradient descent because they are discrete. To solve this problem, we re-parameterize the indices by introducing trainable latent model weight $\mathcal{W} \in \mathbb{R}^{n_{out} \times n_{in}}$ which

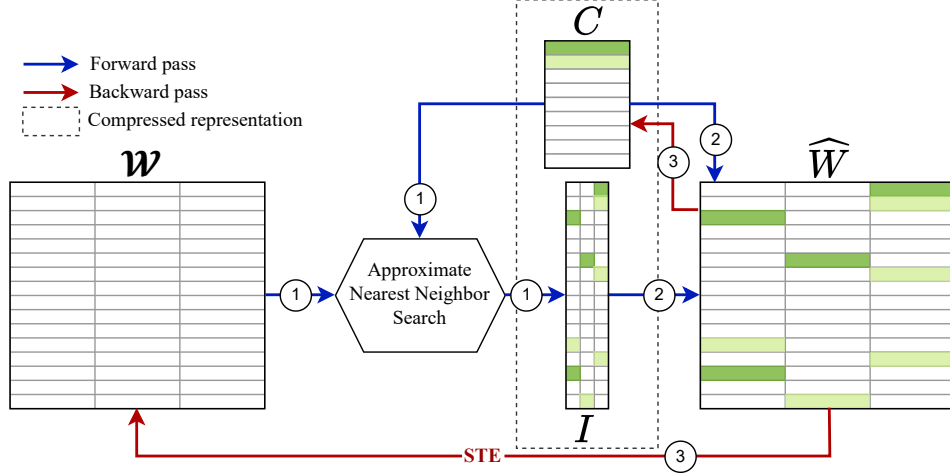


Figure 1: This picture illustrates the proposed method TVQ applied to a single linear layer with the weight matrix W . During optimization, 3 matrices are updated: indices matrix I containing discrete values, the latent weight matrix W and codebook C containing continuous values. TVQ involves 3 main operations: (1) I is calculated using (Approximate) Nearest Neighbor Search 4.2; (2) The dequantized weight matrix $\widehat{W}$ is gathered from C and I by applying the dequantize function 4.1 and is used to calculate the loss function; (3) The gradients are propagated through $\widehat{W}$ to W and C that will be updated during optimization step. After training, only the compressed representation consisting of C and I is stored and used for inference instead of the original weight matrix W .

is initialized with W . We define I as the indices of the nearest codewords to the corresponding vectors ω_{ij}^d of the latent weight W :

$$I = \left\{ I_{ij} \mid I_{ij} = \underset{k}{\operatorname{argmin}} \|\omega_{ij}^d - c_k^d\| \right\} \quad (2)$$

Substituting (2) into the definition the dequantized weight $\widehat{W}$ (1) we reduce it to the nearest neighbor search:

$$\widehat{W} = \left\{ \widehat{\omega}_{ij}^d \mid \widehat{\omega}_{ij}^d = \underset{c_k^d \in C}{\operatorname{argmin}} \|\omega_{ij}^d - c_k^d\| \right\} \quad (3)$$

In simple terms, the latent weight vectors are substituted by their nearest neighbors from the codebook to attain the dequantized weight.

This reparameterization allows us to optimize latent weights W instead of indices matrix I . Note that both W and C are continuous parameters, which allows us to apply gradient descent for their optimization, indirectly enabling the updates for discrete indices according to (2). The gradient for updating W is defined using dequantized weight $\widehat{W}$ and loss function $\mathcal{L}$ as follows:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \widehat{W}} \cdot \frac{\partial \widehat{W}}{\partial W} \quad (4)$$

However, $\partial \widehat{W} / \partial W$ is equal to zero almost everywhere (a.e.) because argmin function used in nearest neighbor search (3) a.e. has zero gradient with

respect to W . According to (4) it results in a.e. zero gradient of $\mathcal{L}$ with respect to W , which makes the standard back-propagation inapplicable.

We propose to use STE to propagate gradients through the nearest neighbor search (3) and obtain a non-zero gradient for W . If the vectors in C are dense enough, then the nearest neighbor search acts like an "almost identity" function of W . For this reason, we replace a.e. zero gradient $\partial \widehat{W} / \partial W$ with the gradient of the identity function. Using this approximation in (4) we obtain the following:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \widehat{W}} \cdot \frac{\partial \widehat{W}}{\partial W} \stackrel{\text{STE}}{=} \frac{\partial \mathcal{L}}{\partial \widehat{W}} \quad (5)$$

STE allows us to pass the gradient through the nearest neighbor search (3) unchanged and use gradient $\partial \mathcal{L} / \partial \widehat{W}$ for the gradient descent updates of W .

The proposed idea has a very simple implementation in PyTorch or TensorFlow frameworks. As described in the Listing 1, we only need to define the forward function for the weight quantization module, which changes the backward function. This module is then used instead of the original weight module in the regular training loop. The general scheme of the method described in this section is presented in Figure 1.

After training, we store a compressed weight representation: the codebook C and the indices matrix I calculated according to (2).

```

1 class VQWeight():
2     def __init__(self, W_fp: tensor,
3                   B: int, d: int):
4         # latent weight init
5         self.W = W_fp
6         # codebook init
7         codebook_size = (2*B, d)
8         self.C = KMeans(W_fp, codebook_size)
9
10    def forward(self):
11        W_hat = NearestNeighbour(self.W, self.C)
12        return W_hat + self.W - self.W.detach()
13
14    def backward(self, grad_output):
15        # automatic by PyTorch

```

Listing 1: PyTorch implementation of TVQ Weight Class. The codebook is initialized with the KMeans algorithm. During the forward pass, we calculate $\widehat{W}$ using the nearest neighbor search function defined in (3). However, because of the forward pass formulation during the backward pass gradient $\partial \mathcal{L} / \partial \mathcal{W}$ is equal to automatically calculated gradient $\partial \mathcal{L} / \partial \widehat{W}$.

4.2 Nearest Neighbor Search

As described in the previous section 4.1, our method involves searching for the nearest vector in the codebook for every vector constituting the latent weight matrix. That requires computing the distance between each vector in the codebook and each vector from the latent weights. The total number of pair-wise distances may reach up to trillions for modern LLMs. Computing them naively presents a challenge from both memory and runtime standpoints and renders the training to be extremely slow.

To solve this problem, we adopt the approximate nearest neighbor search via Voronoi cells (Kolahdouzan and Shahabi, 2004), which is implemented in the open-source library FAISS (Johnson et al., 2019). This approach groups codebook vectors into clusters named Voronoi cells. Then, for each vector from the latent weight it finds several nearest cells and performs an exhaustive nearest neighbor search only within these cells. Thus, a large number of distant vectors is excluded from the search space, and a naive exhaustive nearest neighbor search is performed with the remainder. Our ablation experiments 5.3.2 with 2-bit quantized models show that the approximate search speeds up training many-fold while providing excellent model quality. For 1-bit and 1.5-bit models, we employ an exact (exhaustive) search, as the nearest neighbor search time is small or negligible in these cases due to smaller codebook sizes.

Algorithm 1 Training Pipeline

Require: pretrained model fp_model , input data $\mathbf{D}$, codebook parameters - (B, d) , PTQ iterations T_{ptq} , finetuning iterations T_{ft}

```

1 # Create vector quantized model
2 vq_model = copy(fp_model)
3 for layer ∈ linear_layers(vq_model) do
4     layer.weight = VQWeight(layer.weight, B, d)
5
6 # Block-Wise PTQ with TVQ
7  $\mathbf{X} = \text{fp\_model.input\_embeddings}(\mathbf{D})$ 
8  $\mathbf{X}^q = \text{vq\_model.input\_embeddings}(\mathbf{D})$ 
9 for  $i = 1$  to model.num_blocks do
10     fp_block = fp_model.get_block(i)
11     vq_block = vq_model.get_block(i)
12
13     # Optimize block
14     for  $j = 1$  to  $T_{ptq}$  do
15          $\mathcal{L} = ||\text{fp\_block}(\mathbf{X}_k) - \text{vq\_block}(\mathbf{X}^q_k)||_2$ 
16         #  $k$  - batch index at iteration  $j$ 
17         Propagate gradients using  $\mathcal{L}$ 
18         Update C and  $\mathcal{W}$  with Adam
19
20     # Collect input data for next block
21      $\mathbf{X} = \text{fp\_block}(\mathbf{X})$ 
22      $\mathbf{X}^q = \text{vq\_block}(\mathbf{X}^q)$ 
23
24 # End-to-End Fine-tuning
25 for  $j = 1$  to  $T_{ft}$  do
26      $\mathcal{L} = \text{KL}(\text{vq\_model}(\mathbf{D}_k), \text{fp\_model}(\mathbf{D}_k))$ 
27     Propagate gradients using  $\mathcal{L}$ 
28     Update C with Adam

```

4.3 Training pipeline

Our training pipeline consists of two stages, specifically, Block-Wise Post-Training Quantization with TVQ and End-to-End model fine-tuning. In this section, we detail both stages and provide pseudocode in Algorithm 1.

PTQ stage: Block-Wise Post-Training Vector Quantization with TVQ. In the previous sections, we introduced Trainable Vector Quantization, which optimizes vector-quantized models to minimize arbitrary loss function. We use TVQ in practice to restore the compressed model quality by training VQ models in an efficient block-wise Knowledge Distillation paradigm.

Specifically, we split the quantized model into transformer blocks, which contain a Multi-Head Self-Attention followed by a Multi-Layer Perceptron. We sequentially optimize these blocks with gradient descent according to Section 4.1 to minimize the reconstruction loss function, which aligns the quantized model’s block output activations with those of the uncompressed model, thus distilling knowledge from the original model to the quantized one. For the reconstruction loss that measures the discrepancy between activations, we use L_p distances.

This approach of training model parts sequentially, commonly referred to as Post-Training Quan-

tization (PTQ), became dominant for Scalar LLM quantization due to its limited computational demands (Chen et al., 2024; Xiao et al., 2023; Wei et al., 2022). We are the first to apply the block-wise distillation approach to train VQ models.

E2E stage: End-to-End Fine-tuning. Following prior works, we enhance our method with an End-to-End model fine-tuning. Similar to AQLM and VPTQ fine-tuning, we train only the codebooks, as they contain a relatively small number of parameters, helping to manage memory usage for gradient storage and optimizer states. In theory, the finetuning stage is useful after PTQ as the Knowledge Distillation loss (Geoffrey Hinton and Dean, 2014) based on Kullback–Leibler (KL) divergence computed on the entire model outputs tends to correlate stronger with overall model quality than the reconstruction losses used in PTQ.

5 Experiments

In this section, we report experimental results for the proposed method. We evaluate and compare the TVQ method with other existing approaches on widely used benchmarks for LLMs in a low-bit Post-Training Quantization setting.

5.1 Experimental setup

Model. For our experiments, we employ commonly used open-source Large Language Models, specifically Llama-3-8B, Mistral-7B-v0.1, and Llama-2-7B available on HuggingFace Model Hub (Wolf, 2019). We focus on models with 7–8 billion parameters, as they are feasible for deployment on low-resource devices, such as mobile phones, in the near future. This makes them particularly relevant for real-world applications where memory constraint is a critical factor.

Evaluation. We adopt a standard benchmarking framework to compare our models with existing approaches. Specifically, we report perplexity on the WikiText-2 (Merity et al., 2016) test set (abbreviated in tables as W2) and the validation set of the C4 dataset (Raffel et al., 2020). Additionally, we provide zero-shot accuracy on WinoGrande (Sakaguchi et al., 2021), PiQA (Bisk et al., 2020), HelLaSwag (Zellers et al., 2019), ARC-easy and ARC-challenge Question Answering (QA) benchmarks (Clark et al., 2018), utilizing the Language Model Evaluation Harness library (Gao et al., 2024) for evaluation.

Training Data. In all experiments, we train

our models on the open-source dataset RedPajama (Computer, 2023), following AQLM, EfficientQAT, Quip#. We use the same context lengths as in the original training: 4096 tokens for Llama-2-7B and 8192 tokens for Llama-3-8B and Mistral-7B-v0.1. Models are trained on an equal amount of data, in particular, we sample in total 2^{23} tokens for the PTQ stage and 2^{24} tokens for the fine-tuning stage.

VQ Parameters. For simplicity, we fix the vector size $d = 8$, as in AQLM, Quip#. As for codebook size, we vary this value to get the desired average bit width. Specifically, for approximately 2, 1.5 and 1-bit models we use codebooks of size 2^{15} , 2^{12} , 2^8 respectively.

Training pipeline. We summarize all the hyperparameters for the experiments in the Appendix A.2. Our PTQ experiments can be conducted on a single 32GB V100 GPU. However, if there are more GPUs available, the nearest neighbor search for different layers within the transformer block may be performed concurrently on several devices. In particular, for 2-bit models, parallelizing this computationally costly operation to 2 GPUs speeds up training by 25%. We fine-tune our models using the same hyperparameters as previous works to ensure fair comparisons.

5.2 Experimental results

We compare our TVQ method with existing Vector Quantization methods, described in Section 3, and the state-of-the-art Scalar Quantization method, EfficientQAT (Chen et al., 2024). We use Llama-2-7B as the primary benchmark since all methods report results for this model. Additionally, we evaluate TVQ for Mistral-7B-v0.1 and Llama-3-8B, comparing against methods that provide results for these models. Table 1 shows that TVQ outperforms existing methods in all experiments. For Llama-3-8B, we surpass the closest competitor, Quip# by 2.1% on average across QA benchmarks. For Llama-2-7B, our 2-bit model is only 2.4% worse than the uncompressed model, significantly narrowing the performance gap.

A particularly interesting comparison is with AQLM, which shares the same VQ structure and codebook training pipeline as TVQ, differing only in index optimization approach. TVQ outperforms AQLM by 1.5% on Mistral-7B-v0.1 and 1.7% for Llama-2-7B on average across QA benchmarks, bridging the gap to the uncompressed Llama-2-7B to 2.4% only. Notably, our TVQ method out-

Method	Bits	W2↓	C4↓	WinoG↑	PiQA↑	HellaS ↑	ArcE↑	ArcC↑					
Llama-3-8B													
FP16	16	5.54	7.10	73.1	79.6	60.2	80.1	50.3					
EfficientQAT	2.25	-	-	65.7	75.3	50.7	69.2	36.0					
QuIP#	2.02	7.84	-	68.2	75.6	50.7	72.9	39.2					
VPTQ	2.02	-	-	65.9	75.1	52.2	71.0	36.9					
TVQ (Ours)	2.01	7.59	8.86	71.5	76.6	54.3	73.5	41.3					
Mistral-7B-v0.1													
FP16	16	4.77	5.71	73.6	80.5	61.2	78.9	49.2					
AQLM	2.01	5.76	6.60	68.7	77.6	56.4	73.3	42.7					
VPTQ	2.04	5.64	6.43	68.7	77.9	56.1	72.2	41.1					
TVQ (Ours)	2.01	5.62	6.42	70.4	78.0	57.6	76.4	43.5					
Llama-2-7B													
		results with *		t _{old}	t _{cur}	t _{old}	t _{cur}	t _{old}	t _{cur}	t _{old}	t _{cur}	t _{old}	t _{cur}
FP16	16	5.12	6.63	67.3	69.2	78.5	78.0	56.7	57.2	69.3	76.3	40.0	43.5
EfficientQAT	2.25	7.19	8.79	-	66.2	-	74.2	-	50.8	-	69.8	-	36.5
ReALLM	2	6.69	8.28	-	66.5	-	75.7	-	-	-	68.6	-	35.2
VPTQ	2.02	6.13	8.07	64.3	65.9	75.2	75.5	52.1	52.8	63.8	70.1	35.2	38.3
AQLM	2.02	6.14	8.09	65.7	-	76.0	-	51.8	-	63.4	-	34.4	-
QuIP#	2.02	6.19	8.16	65.0	-	75.4	-	51.9	-	65.0	-	35.2	-
TVQ (Ours)	2.02	6.11	7.99	65.4	67.6	76.3	75.8	52.8	53.0	67.7	72.8	37.5	39.4

Table 1: Evaluation of quantized Llama-2-7B, Llama-3-8B and Mistral-7B-v0.1 models. The table reports perplexity measured on the WikiText-2 test set (W2) and C4 validation set, as well as accuracy for zero-shot tasks: WinoGrande (WinoG), PiQA, HellaSwag (HellaS), ARC-easy (ArcE) and ARC-challenge (ArcC) question answering benchmarks. Similar to previous works, we use a context length of 4096 for the Llama-2-7B model and a context length of 8192 for both Mistral-7B-v0.1 and Llama-3-8B models. *For the uncompressed FP16 Llama-2-7B model results, two different versions of evaluation on QA benchmarks exist in the literature. These differences are caused by the inconsistency of handling special tokens in the transformers library. To ensure a fair method comparison for the Llama-2-7B model, we have evaluated both versions. To distinguish between old and current versions of tokenizers we denote them as **t**_{old} and **t**_{cur} respectively.

performs VPTQ on Mistral-7B-v0.1, even though VPTQ benefits from an additional salient weight codebook designed to simplify quantization.

Method	Bits	W2↓	C4↓	Avg.↑
FP16	16	5.12	6.63	67.3
CRVQ	1.07	13.7	15.5	46.0
TVQ (Ours)	1.00	12.6	15.0	47.1
TVQ (Ours)	1.52	7.32	9.38	56.8

Table 2: Evaluation metrics of extremely low-bit quantized Llama-2-7B with 4096 context length. Avg. refers to the average over QA tasks defined in Section 5.1.

Finally, in Table 2 we present results in extremely low-bit settings (1-bit and 1.5-bit) for Llama-2-7B. Research of low-bit VQ remains limited, with only CRVQ reporting results for 1-bit models. Our TVQ 1-bit model outperforms CRVQ, which builds on AQLM by introducing salient

weight structures by a large margin of 1.1 Wiki-text PPL.

5.3 Ablation Study

We devote ablation analysis to examine the key aspects of the proposed method: the necessity of simultaneous training of latent weights $\mathcal{W}$ and codebook $\mathcal{C}$, choice of parameters for approximate neighbor search and impact of PTQ and E2E training stages on the final performance.

5.3.1 Indices Optimization

TVQ is designed for indices optimization through a reparameterization defined in (2) and trainable latent weight $\mathcal{W}$ with an optionally trainable codebook $\mathcal{C}$. To assess the contribution of TVQ’s components, we conduct ablation experiment with the following setups:

1. Indices remain fixed, $\mathcal{C}$ is trained.
2. Indices are updated via reparameterization, $\mathcal{C}$ is trained while $\mathcal{W}$ remains fixed at the initial

uncompressed weight value W .

3. Indices are updated via reparameterization, $\mathcal{W}$ is trained while C remains fixed.
4. Indices are updated via reparameterization, both $\mathcal{W}$ and C are trained.

We evaluate VQ model quality for these strategies in Table 4. A key takeaway is that latent weight training is crucial for all compression rates, as the first two setups yield significantly worse results than the last two. Additionally, codebook training provides only marginal improvements, as setups 3 and 4 show little difference. This result indicates that TVQ remains effective even with a fixed codebook, making TVQ particularly suitable for hardware-efficient designs of codebooks, such as lattice codebooks from Quip#.

5.3.2 Approximate Nearest Neighbor Search

We analyze the impact of the approximate nearest neighbor search on the final model quality. We conduct experiments with varying parameters that affect search quality, particularly the number of Voronoi cells N and the number of cells for exhaustive search K . Search quality is evaluated using accuracy, which is defined as the percentage of queries where the approximate search matches the exact search. Additionally, we measure speedup by comparing the approximate search against an exact search implemented on a single 32GB V100 GPU.

As shown in Table 3, an approximate search achieving 99.99% accuracy is five times faster than the exact search. Model quality remains unaffected even when search accuracy drops to 95%, and the decline is minimal even at 80% accuracy. These findings suggest that opting for approximate search over exact search is unlikely to degrade model quality.

Search Accuracy	N	K	Speedup	W2↓	C4↓
99.9 %	256	16	×5.0	6.70	8.70
99 %	1024	16	×5.5	6.64	8.68
98 %	256	8	×6.4	6.63	8.69
95 %	512	8	×6.5	6.68	8.73
80 %	1024	4	×6.7	6.84	8.92
60 %	1024	2	×6.7	7.13	9.15

Table 3: Ablation study of Approximate Nearest Neighbor search (5.3.2) for 2 bit Llama-2-7B. N denotes the number of Voronoi cells, K — the number of visited cells. The grey line highlights the selected setting. Table shows evaluation metrics for 2 bit Llama-2-7B model after PTQ stage, without E2E stage.

Update			2.02 bits		1.52 bits		1.00 bits	
I	C	$\mathcal{W}$	W2↓	C4↓	W2↓	C4↓	W2↓	C4↓
×	✓	×	7.23	9.39	17.4	18.6	>1k	>1k
✓	✓	×	7.38	9.42	269	199	>1k	>1k
✓	×	✓	6.80	8.95	9.54	11.9	23.2	25.6
✓	✓	✓	6.63	8.69	9.29	11.6	26.7	28.0

Table 4: Ablation study of optimized parameters. Each row refers to one of the optimization strategies described in 5.3.1: only codebook C training (first row), experiments with indices matrix I reparameterization and trainable codebook C (second row), trainable latent weight $\mathcal{W}$ (third row), both parameters C and $\mathcal{W}$ are trainable (last row). Table shows evaluation metrics for quantized Llama-2-7B models after PTQ stage, without E2E stage.

5.3.3 Training pipeline

In Table 5, we analyze the contribution of the training stages described in Section 4.3 to the final model quality. Evidently, the two-staged pipeline significantly outperforms the PTQ-only and E2E-only stages. Notably, in the extremely low-bit setting of 1 bit, the PTQ stage alone achieves substantially better results than E2E fine-tuning. This highlights the importance of the proposed index updates, which occur only during the PTQ stage.

Pipeline		2.02 bits		1.52 bits		1.00 bits	
PTQ	E2E	W2↓	C4↓	W2↓	C4↓	W2↓	C4↓
×	✓	6.41	8.41	8.74	11.2	203	141
✓	×	6.63	8.69	9.29	11.6	26.7	28.0
✓	✓	6.11	7.99	7.33	9.38	12.6	15.0

Table 5: Ablation study of training pipeline (5.3.3) for quantized Llama-2-7B models with 4096 context length.

6 Conclusions & Discussion

In this work, we tackled the index optimization problem in VQ models, introducing the first algorithm that enables gradient-based index optimization. Our approach outperforms existing methods across popular LLM benchmarks. Moreover, TVQ is straightforward to implement using popular deep learning frameworks, facilitating real-life adoption. While our primary focus is index optimization, integrating TVQ with existing VQ-friendly structures such as salient weight tables and incoherence processing has the potential to further enhance quality and compression rates. We believe future research can leverage TVQ for index optimization while advancing VQ architectures, further improving LLM efficiency.

7 Limitations

Several works, such as VPTQ and AQLM, provide software tools for faster execution on modern hardware, such as CUDA kernels for NVIDIA GPUs. These works have shown that vector quantization not only saves memory but also significantly benefits throughput and latency. One limitation of our work is that we do not contribute to the further development of hardware-specific utilities for vector quantization. Nevertheless, the structure of our vector-quantized models is identical to those introduced before, and therefore, our models can be executed using the software tools released in the existing works, gaining equal efficiency and speed up as existing checkpoints.

References

- Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. Tensorflow: a system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation, OSDI'16*, page 265–283, USA. USENIX Association.
- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. [arXiv preprint arXiv:2303.08774](https://arxiv.org/abs/2303.08774).
- Yoshua Bengio, Nicholas Leonard, and Aaron Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. [arXiv preprint arXiv:1308.3432](https://arxiv.org/abs/1308.3432).
- Yonatan Bisk, Rowan Zellers, Jianfeng Gao, and Yejin Choi. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439.
- Mengzhao Chen, Wenqi Shao, Peng Xu, Jiahao Wang, Peng Gao, Kaipeng Zhang, Yu Qiao, and Ping Luo. 2024. Efficientqat: Efficient quantization-aware training for large language models. [arXiv preprint arXiv:2407.11062](https://arxiv.org/abs/2407.11062).
- Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. Pact: Parameterized clipping activation for quantized neural networks. [arXiv preprint arXiv:1805.06085](https://arxiv.org/abs/1805.06085).
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. [arXiv preprint arXiv:1803.05457](https://arxiv.org/abs/1803.05457).
- Together Computer. 2023. [Redpajama: An open dataset for training large language models](https://redpajama.com/).
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2024. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36.
- Xuanyi Dong and Yi Yang. 2019. Searching for a robust neural architecture in four gpu hours. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1761–1770.
- A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, and A. Goyal. 2024. The llama 3 herd of models. [arXiv preprint arXiv:2407.21783](https://arxiv.org/abs/2407.21783).
- Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. Extreme compression of large language models via additive quantization. In *International Conference on Machine Learning*.
- Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. 2020. [Learned step size quantization](https://arxiv.org/abs/2006.06732). In *International Conference on Learning Representations*.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2024. [A framework for few-shot language model evaluation](https://arxiv.org/abs/2406.12783).
- Oriol Vinyals Geoffrey Hinton and Jeff Dean. 2014. Distilling the knowledge in a neural network. In *NeurIPS Deep Learning Workshop*.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. [arXiv preprint arXiv:2501.12948](https://arxiv.org/abs/2501.12948).
- Song Han, Huizi Mao, and William J Dally. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. [arXiv preprint arXiv:1510.00149](https://arxiv.org/abs/1510.00149).
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. [LoRA: Low-rank adaptation of large language models](https://arxiv.org/abs/2206.01068). In *International Conference on Learning Representations*.

681	Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2018. Quantized neural networks: Training neural networks with low precision weights and activations. <i>Journal of Machine Learning Research</i> , 18(187):1–30.	738
682		739
683		740
684		741
685		742
686	Eric Jang, Shixiang Gu, and Ben Poole. 2017. Categorical reparameterization with gumbel-softmax . In <i>International Conference on Learning Representations</i> .	743
687		744
688		745
689		746
690	Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. 2023. Mistral 7b. <i>arXiv preprint arXiv:2310.06825</i> .	747
691		748
692		749
693		750
694		751
695	Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. <i>IEEE Transactions on Big Data</i> , 7(3):535–547.	752
696		753
697		
698	Mohammad Kolahdouzan and Cyrus Shahabi. 2004. Voronoi-based k nearest neighbor search for spatial network databases. In <i>Proceedings of the Thirtieth international conference on Very large data bases-Volume 30</i> , pages 840–851.	754
699		755
700		756
701		757
702		758
703	Louis Leconte, Lisa Bedin, Van Minh Nguyen, and Eric Moulines. 2024. Reallm: A general framework for llm compression and fine-tuning. <i>arXiv preprint arXiv:2405.13155</i> .	759
704		760
705		761
706		762
707		763
708	Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In <i>International Conference on Machine Learning</i> , pages 19274–19286. PMLR.	764
709		765
710		766
711	Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. 2021. Brecq: Pushing the limit of post-training quantization by block reconstruction. In <i>International Conference on Learning Representations</i> .	767
712		768
713		769
714		770
715		771
716	Yifei Liu, Jicheng Wen, Yang Wang, Shengyu Ye, Li Lina Zhang, Ting Cao, Cheng Li, and Mao Yang. 2024. VPTQ: Extreme Low-bit Vector Post-Training Quantization for Large Language Models. In <i>Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing</i> , pages 8181–8196. Association for Computational Linguistics.	772
717		773
718		774
719		775
720		776
721		777
722		
723		
724	Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. <i>arXiv preprint arXiv:1609.07843</i> .	778
725		779
726		780
727	Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. Pytorch: An imperative style, high-performance deep learning library . In <i>Advances in Neural Information Processing Systems</i> , volume 32. Curran Associates, Inc.	781
728		782
729		783
730		784
731		785
732		
733		
734		
735		
736		
737		
	Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. <i>The Journal of Machine Learning Research</i> , 21(1):5485–5551.	786
		787
		788
	Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. <i>Communications of the ACM</i> , 64(9):99–106.	
	Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, and et al. 2023. Llama 2: Open foundation and fine-tuned chat models. <i>arXiv preprint arXiv:2307.09288</i> .	
	Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. 2024. Quip: Even better LLM quantization with hadamard incoherence and lattice codebooks . In <i>International Conference on Machine Learning</i> .	
	Antoine Vanderschueren and Christophe De Vleeschouwer. 2023. Are straight-through gradients and soft-thresholding all you need for sparse training? In <i>Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision</i> , pages 3808–3817.	
	Xiuying Wei, Ruihao Gong, Yuhang Li, Xianglong Liu, and Fengwei Yu. 2022. Qdrop: Randomly dropping quantization for extremely low-bit post-training quantization. In <i>International Conference on Learning Representations</i> .	
	T Wolf. 2019. Huggingface’s transformers: State-of-the-art natural language processing. <i>arXiv preprint arXiv:1910.03771</i> .	
	Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In <i>International Conference on Machine Learning</i> , pages 38087–38099. PMLR.	
	Yuzhuang Xu, Shiyu Ji, Qingfu Zhu, and Wanxiang Che. 2024. Crvq: Channel-relaxed vector quantization for extreme compression of llms. <i>arXiv preprint arXiv:2412.09282</i> .	
	Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? <i>arXiv preprint arXiv:1905.07830</i> .	
	B Zoph. 2016. Neural architecture search with reinforcement learning. <i>arXiv preprint arXiv:1611.01578</i> .	

A Training details

A.1 Weight Preprocessing

As in AQLM and Quip#, we scale weight matrix output channels to reduce outliers. Specifically, before VQ, we divide the elements of rows of W by the scale factors $s_i = \|W_{i,:}\|_2 \in \mathbb{R}^{n_{out}}$, during dequantization, the inverse scaling can be effectively applied after the activation and weight matrix multiplication, introducing a negligible (less than 0.01 bits per weight) memory overhead.

A.2 Training hyperparameters

This section summarizes the hyperparameters used in all our experiments for both Block-Wise PTQ and E2E fine-tuning stages.

For Block-Wise PTQ, we used the Adam optimizer with default weight decay and beta values and the Cosine Annealing scheduler. For End-to-End fine-tuning, we used the Adam optimizer with weight decay equal to 0.1 and betas equal to 0.9 and 0.95 respectively. We fine-tuned all models for 1 epoch. All other relevant training parameters are summarized in the Table 6.

B Additional results for low-bit Llama-2-7B

In the experimental section, we have provided the results for our low-bit models, indicating only the average over QA tasks. Table 7 demonstrates detailed results for each task used in evaluation.

Model	Bits	PTQ stage							E2E stage			
		SL	SN	E	BS	LR_W	LR_C	Loss	SL	SN	BS	LR_C
Llama-2-7B	1	4096	2048	3	4	1e-4	1e-4	L_2	4096	4096	8	3e-5
Llama-2-7B	1.52	4096	2048	2	4	5e-5	5e-5	L_2	4096	4096	8	3e-5
Llama-2-7B	2.02	4096	2048	2	8	5e-5	5e-5	L_1	4096	4096	8	1e-5
Llama-3-8B	2.01	8192	1024	2	8	5e-5	15e-5	L_1	8192	2048	4	1e-5
Mistral-7B-v0.1	2.01	8192	1024	2	8	2e-5	4e-5	L_1	8192	2048	4	1e-5

Table 6: The table presents training parameters for all experiments in the article, in particular, sequence length (SL), number of training epoch (E), batch size (BS), learning rate for the latent weights (LR_W), learning rate for the codebooks (LR_C), reconstruction loss function for PTQ stage (Loss). For E2E fine-tuning, the number of epochs is defaulted to 1.

Method	Bits	W2↓	C4↓	WinoG↑	PiQA↑	HellaS ↑	ArcE↑	ArcC↑
FP16	16	5.12	6.63	69.2	78.0	57.2	76.3	43.5
TVQ (Ours)	1.52	7.32	9.38	62.3	73.3	48.8	66.5	32.9
TVQ (Ours)	1.00	12.6	15.0	57.3	66.1	37.5	50.1	24.4

Table 7: Evaluation of 1.52 and 1 bits quantized Llama-2-7B models. The table reports perplexity measured on the WikiText-2 test set (W2) and C4 validation set, as well as accuracy for zero-shot tasks: WinoGrande (WinoG), PiQA, HellaSwag (HellaS), ARC-easy (ArcE) and ARC-challenge (ArcC) question answering benchmarks, measured with the latest tokenizer versions. Similar to previous works, we use a context length of 4096 for the Llama-2-7B model.